

# POSTGRESQL 18 DBA

## End-to-End Basic Terminology Interview Cheat Sheet

Installation • Configuration • Architecture • Storage • Security • WAL • MVCC • VACUUM • Performance •  
Backup • PITR • Replication • HA • Monitoring • Upgrades

### Technical Interview Revision Guide

Technical baseline: PostgreSQL 18.6 Official Documentation

## Table of Contents

| Section                                     | Page |
|---------------------------------------------|------|
| 1. PostgreSQL Fundamentals                  | 3    |
| 2. Installation and Server Setup            | 3    |
| 3. Configuration Files and Parameters       | 4    |
| 4. Authentication, Roles, and Security      | 5    |
| 5. PostgreSQL Process Architecture          | 6    |
| 6. Storage Architecture                     | 7    |
| 7. Memory and Connection Management         | 7    |
| 8. WAL, Checkpoints, and Reliability        | 8    |
| 9. MVCC and Transactions                    | 9    |
| 10. VACUUM and Routine Maintenance          | 10   |
| 11. Query Planning, Statistics, and Indexes | 10   |
| 12. Partitioning and Database Objects       | 11   |
| 13. Backup, Restore, and PITR               | 12   |
| 14. Physical and Logical Replication        | 13   |
| 15. High Availability and Recovery Concepts | 14   |
| 16. Monitoring and Troubleshooting          | 15   |
| 17. Core DBA Utilities                      | 16   |
| 18. Upgrades and Lifecycle                  | 17   |
| Official PostgreSQL References              | 18   |

## 1. PostgreSQL Fundamentals

| Term                     | Simple Meaning                                                      | Interview / DBA Point                                                                                                          |
|--------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>PostgreSQL</b>        | Open-source object-relational database management system.           | Mention ACID, MVCC, WAL, extensibility, replication, and SQL support.                                                          |
| <b>Postgres</b>          | Common short name for PostgreSQL.                                   | PostgreSQL is the official project/product name; Postgres is widely used shorthand.                                            |
| <b>Server / Instance</b> | A running PostgreSQL server process managing a database cluster.    | One server instance manages one database cluster/data directory at a time.                                                     |
| <b>Database Cluster</b>  | Collection of databases managed by one PostgreSQL server instance.  | In PostgreSQL, cluster does not automatically mean an HA cluster.                                                              |
| <b>Database</b>          | Logical container for schemas and database objects.                 | Connections are made to a database; cross-database queries are not native like cross-database three-part naming in SQL Server. |
| <b>Schema</b>            | Namespace inside a database used to organize objects.               | The same object name can exist in different schemas.                                                                           |
| <b>Object</b>            | General term for tables, indexes, views, sequences, functions, etc. | Ownership and privileges are object-specific.                                                                                  |
| <b>Relation</b>          | PostgreSQL term used broadly for table-like/storage objects.        | You will see relation terminology in catalogs, locks, sizes, and errors.                                                       |
| <b>Tuple</b>             | A row or row version.                                               | MVCC can create multiple tuple versions for one logical row.                                                                   |
| <b>Page / Block</b>      | Basic on-disk page unit; normally 8 KB with the default build.      | Performance discussions often refer to block reads/hits and page-level storage.                                                |
| <b>Session</b>           | A client's active connection context.                               | One connection normally maps to one backend process.                                                                           |
| <b>Transaction</b>       | One or more operations treated as a logical unit of work.           | Core commands: BEGIN, COMMIT, ROLLBACK, SAVEPOINT.                                                                             |

## 2. Installation and Server Setup

| Term                        | Simple Meaning                                          | Interview / DBA Point                                                                                         |
|-----------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>PostgreSQL Binaries</b>  | Executable programs installed with PostgreSQL.          | Know postgres, psql, pg_ctl, initdb, pg_dump, pg_restore, pg_basebackup, pg_upgrade.                          |
| <b>Package Installation</b> | Installing PostgreSQL using OS packages/repositories.   | Operationally easier for patching and service integration than source builds in most enterprise environments. |
| <b>Source Installation</b>  | Building PostgreSQL from source code.                   | Useful for custom compile options, but requires more lifecycle management.                                    |
| <b>initdb</b>               | Initializes a new PostgreSQL database cluster.          | Creates system catalogs, initial databases, control files, configuration, and storage structure.              |
| <b>PGDATA</b>               | Common environment name for the cluster data directory. | Always verify the actual data_directory rather than assuming a path.                                          |

| Term                         | Simple Meaning                                                                    | Interview / DBA Point                                                                                |
|------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Data Directory</b>        | Physical directory containing the database cluster storage and control structure. | Check with SHOW data_directory; configuration files can be placed elsewhere.                         |
| <b>PostgreSQL OS Account</b> | Dedicated operating-system account that owns/runs the server.                     | On many Linux packages it is postgres; PostgreSQL should not run as root.                            |
| <b>Port</b>                  | TCP port on which PostgreSQL listens; default is 5432.                            | Verify using SHOW port and OS socket/listening tools.                                                |
| <b>Service Management</b>    | Starting, stopping, restarting, or reloading the server.                          | On Linux this is commonly integrated with systemd; pg_ctl is PostgreSQL's native control utility.    |
| <b>Startup</b>               | Process of launching postgres and recovery/startup work if required.              | Startup failure often points to configuration, permissions, port, shared memory, or recovery issues. |
| <b>Shutdown Modes</b>        | Smart, fast, and immediate shutdown behavior.                                     | Fast is common for administration; immediate requires crash recovery on next start.                  |
| <b>Data Checksums</b>        | Optional page checksums used to detect certain storage corruption.                | PostgreSQL 18 initdb enables data checksums by default unless disabled.                              |

**Quick commands**

```

SELECT version();
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
SHOW port;
SHOW listen_addresses;

```

### 3. Configuration Files and Parameters

| Term                        | Simple Meaning                                                                         | Interview / DBA Point                                                                            |
|-----------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>postgresql.conf</b>      | Primary server configuration file.                                                     | Controls connections, memory, WAL, checkpoints, logging, autovacuum, planner and more.           |
| <b>pg_hba.conf</b>          | Host-Based Authentication rules file.                                                  | Controls who can connect to which database, from where, and using which authentication method.   |
| <b>pg_ident.conf</b>        | Maps external/OS user names to PostgreSQL roles for applicable authentication methods. | Often used with peer/certificate-style identity mapping scenarios.                               |
| <b>postgresql.auto.conf</b> | Configuration file written by ALTER SYSTEM.                                            | Do not treat it as a general hand-edit file; ALTER SYSTEM settings can override postgresql.conf. |
| <b>pg_settings</b>          | System view exposing current parameter values and metadata.                            | Check context, unit, source, pending_restart and sourcefile when troubleshooting settings.       |
| <b>SHOW</b>                 | Displays one or all configuration values.                                              | Example: SHOW shared_buffers;                                                                    |
| <b>SET</b>                  | Changes a setting for the current session when allowed.                                | Useful for safe session-level testing of work_mem and planner options.                           |
| <b>ALTER SYSTEM</b>         | Writes cluster-wide settings to postgresql.auto.conf.                                  | Reload or restart depending on the parameter context.                                            |

| Term                     | Simple Meaning                                                                 | Interview / DBA Point                                                                              |
|--------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Reload</b>            | Re-reads reloadable configuration without full server restart.                 | Use <code>SELECT pg_reload_conf();</code> or <code>service/pg_ctl reload</code> .                  |
| <b>Restart</b>           | Stops and starts PostgreSQL to apply postmaster-context settings.              | <code>shared_buffers</code> and <code>max_connections</code> are common restart-required examples. |
| <b>Parameter Context</b> | Defines when/how a parameter can change.                                       | Know internal, postmaster, sighup, backend, superuser-backend, superuser, user.                    |
| <b>listen_addresses</b>  | Controls server interfaces/IP addresses that accept TCP connections.           | Must be combined with <code>pg_hba.conf</code> and network/firewall access.                        |
| <b>max_connections</b>   | Maximum configured concurrent database connections, subject to reserved slots. | Large values increase process and memory overhead; pooling is often preferred.                     |

**Quick commands**

```
SHOW shared_buffers;
SELECT name, setting, unit, context, source, pending_restart FROM pg_settings ORDER BY name;
ALTER SYSTEM SET work_mem = '16MB';
SELECT pg_reload_conf();
```

## 4. Authentication, Roles, and Security

| Term                 | Simple Meaning                                                                     | Interview / DBA Point                                                                                                                                                                   |
|----------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Role</b>          | PostgreSQL identity used for ownership and privileges.                             | Roles can behave as users, groups, or both.                                                                                                                                             |
| <b>User</b>          | A role with LOGIN capability.                                                      | PostgreSQL does not maintain a completely separate user object model.                                                                                                                   |
| <b>Superuser</b>     | Highly privileged role that bypasses most permission checks.                       | Limit usage and use least privilege.                                                                                                                                                    |
| <b>LOGIN</b>         | Role attribute allowing database login.                                            | <code>CREATE ROLE appuser LOGIN;</code>                                                                                                                                                 |
| <b>CREATEDB</b>      | Role attribute permitting database creation.                                       | Separate it from superuser privileges when possible.                                                                                                                                    |
| <b>CREATEROLE</b>    | Role attribute permitting role administration within PostgreSQL rules.             | Powerful privilege; do not grant broadly.                                                                                                                                               |
| <b>REPLICATION</b>   | Role attribute used for physical/logical replication connections where applicable. | Replication accounts should be tightly restricted in <code>pg_hba.conf</code> .                                                                                                         |
| <b>GRANT</b>         | Grants object privileges or role membership.                                       | Know <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>USAGE</code> , <code>EXECUTE</code> , <code>CONNECT</code> and <code>CREATE</code> . |
| <b>REVOKE</b>        | Removes privileges or role membership.                                             | Remember privileges can come through <code>PUBLIC</code> or inherited roles.                                                                                                            |
| <b>Ownership</b>     | Each database object has an owner.                                                 | Owners have inherent control and can grant privileges.                                                                                                                                  |
| <b>SCRAM-SHA-256</b> | Modern PostgreSQL password                                                         | Prefer SCRAM over older MD5-style password                                                                                                                                              |

| Term                            | Simple Meaning                                                                     | Interview / DBA Point                                                                       |
|---------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
|                                 | authentication method.                                                             | authentication for modern deployments.                                                      |
| <b>Peer Authentication</b>      | Uses the local operating-system identity for authentication.                       | Common for local administrative access on Linux packages.                                   |
| <b>Trust Authentication</b>     | Allows matching connections without a password.                                    | Use only in tightly controlled scenarios; generally inappropriate for broad network access. |
| <b>SSL/TLS</b>                  | Encrypts client/server network traffic and can support certificate authentication. | hostssl rules require SSL-capable connections.                                              |
| <b>Row-Level Security (RLS)</b> | Policies can restrict which rows a role can see or modify.                         | Useful for multi-tenant and fine-grained access control.                                    |

## 5. PostgreSQL Process Architecture

| Term                                      | Simple Meaning                                                             | Interview / DBA Point                                                                        |
|-------------------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Postmaster / Main postgres Process</b> | Main server process that listens for clients and manages server processes. | Postmaster is historical terminology; process listings normally show postgres.               |
| <b>Backend Process</b>                    | Server process handling a client session.                                  | PostgreSQL traditionally uses a process-per-connection model.                                |
| <b>Auxiliary Process</b>                  | Core server helper process performing internal work.                       | Examples include checkpointer, background writer, WAL writer, autovacuum launcher, archiver. |
| <b>Checkpointer</b>                       | Coordinates checkpoints and required buffer flushing.                      | Checkpoints bound crash-recovery work and influence write I/O patterns.                      |
| <b>Background Writer</b>                  | Writes dirty shared buffers gradually.                                     | Helps reduce writes that backends would otherwise perform themselves.                        |
| <b>WAL Writer</b>                         | Writes WAL buffers to WAL files periodically.                              | Supports timely WAL write-out outside individual backend activity.                           |
| <b>Autovacuum Launcher</b>                | Launches autovacuum workers based on maintenance needs.                    | One key coordinator for automatic VACUUM/ANALYZE.                                            |
| <b>Autovacuum Worker</b>                  | Performs automatic VACUUM/ANALYZE work.                                    | Table-level settings can override cluster defaults.                                          |
| <b>Archiver</b>                           | Archives completed WAL segments when archiving is enabled.                 | Failures can cause pg_wal growth and break PITR objectives.                                  |
| <b>WAL Sender</b>                         | Primary-side process that streams WAL to replication consumers.            | Visible in pg_stat_replication for physical replication.                                     |
| <b>WAL Receiver</b>                       | Standby-side process receiving WAL from upstream.                          | Monitor via pg_stat_wal_receiver.                                                            |
| <b>Startup Process</b>                    | Performs recovery/WAL replay on standby or after crash.                    | A standby remains in recovery until promoted.                                                |
| <b>WAL Summarizer</b>                     | Creates WAL summary information used for incremental backup support.       | Important PostgreSQL 18 terminology.                                                         |

## 6. Storage Architecture

| Term                        | Simple Meaning                                                                   | Interview / DBA Point                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Heap</b>                 | Main table storage containing row versions.                                      | Heap tables are separate from indexes.                                                                          |
| <b>Relation File</b>        | Physical file(s) backing a relation.                                             | Large relations are segmented into multiple files internally.                                                   |
| <b>Tablespace</b>           | Location allowing objects to be stored outside the default data directory.       | Nearest SQL Server concept is filegroup, but it is not equivalent.                                              |
| <b>TOAST</b>                | Mechanism for compressing and/or storing oversized attribute values out-of-line. | Important for large text, bytea, JSONB and other wide values.                                                   |
| <b>Free Space Map (FSM)</b> | Tracks free space available in relation pages.                                   | Helps PostgreSQL find pages suitable for inserts/updates.                                                       |
| <b>Visibility Map (VM)</b>  | Tracks pages known all-visible and/or all-frozen.                                | Important for VACUUM behavior and index-only scans.                                                             |
| <b>OID</b>                  | Object Identifier used internally for many catalog objects.                      | Useful when reading system catalogs; not every user table has an OID column.                                    |
| <b>System Catalog</b>       | Internal metadata tables, primarily in pg_catalog.                               | Key catalogs include pg_class, pg_attribute, pg_database, pg_namespace, pg_index.                               |
| <b>information_schema</b>   | SQL-standard metadata views.                                                     | More portable than pg_catalog but less PostgreSQL-specific.                                                     |
| <b>Unlogged Table</b>       | Table whose changes are not WAL-logged in the normal durable way.                | Faster for some workloads but contents are not crash-safe and are not physically replicated like normal tables. |
| <b>Temporary Table</b>      | Session- or transaction-scoped table stored as temporary data.                   | Visibility and lifecycle differ from permanent tables.                                                          |

## 7. Memory and Connection Management

| Term                        | Simple Meaning                                                                  | Interview / DBA Point                                                                   |
|-----------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>shared_buffers</b>       | PostgreSQL's primary shared buffer cache.                                       | Not total database memory; PostgreSQL also relies on OS cache and per-operation memory. |
| <b>work_mem</b>             | Memory limit used by individual query operations before spilling to temp files. | Not simply per connection; one query can have multiple work_mem-consuming nodes.        |
| <b>maintenance_work_mem</b> | Memory for maintenance operations such as VACUUM and CREATE INDEX.              | Can be much larger than work_mem when sizing allows.                                    |
| <b>autovacuum_work_mem</b>  | Memory limit for autovacuum workers.                                            | If set to -1, maintenance_work_mem is used.                                             |
| <b>wal_buffers</b>          | Shared memory for WAL records before WAL file write-out.                        | Usually left to PostgreSQL automatic sizing unless there is a clear reason.             |
| <b>effective_cache_size</b> | Planner estimate of cache available                                             | It does not allocate memory.                                                            |

| Term                         | Simple Meaning                                                                 | Interview / DBA Point                                                                 |
|------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|                              | from PostgreSQL + OS.                                                          |                                                                                       |
| <b>huge_pages</b>            | Controls use of OS huge pages for PostgreSQL shared memory where supported.    | Can reduce page-table overhead in larger systems.                                     |
| <b>Connection Pooling</b>    | Reuses a smaller number of database connections for more application requests. | Useful because PostgreSQL uses a process-per-connection model.                        |
| <b>PgBouncer</b>             | Popular external PostgreSQL connection pooler.                                 | It is not part of PostgreSQL core.                                                    |
| <b>Connection Exhaustion</b> | All normal connection slots are consumed.                                      | Investigate pg_stat_activity, application pooling, idle sessions and max_connections. |
| <b>Idle Session</b>          | Connected session not currently running a query.                               | Usually lower risk than idle-in-transaction.                                          |
| <b>Idle in Transaction</b>   | Session with an open transaction but currently idle.                           | Can hold locks/snapshots and prevent cleanup; often operationally dangerous.          |

## 8. WAL, Checkpoints, and Reliability

| Term                                | Simple Meaning                                                                   | Interview / DBA Point                                                                                  |
|-------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>WAL</b>                          | Write-Ahead Logging used for durability and recovery.                            | Required WAL must reach durable storage before corresponding dirty data pages can be safely persisted. |
| <b>WAL Record</b>                   | Individual recovery record describing a change.                                  | Recovery replays WAL records to reach a consistent state.                                              |
| <b>WAL Segment</b>                  | File containing WAL records; default segment size is normally 16 MB.             | Segment size is selected at initdb time.                                                               |
| <b>pg_wal</b>                       | Directory under PGDATA containing active/retained WAL.                           | Never manually delete WAL files to solve disk pressure.                                                |
| <b>LSN</b>                          | Log Sequence Number identifying a position in WAL.                               | Core unit for replication, recovery, backup and lag comparisons.                                       |
| <b>Checkpoint</b>                   | Recovery point that flushes required dirty buffers and records checkpoint state. | Too-frequent checkpoints can increase I/O; too-infrequent can increase recovery time/WAL volume.       |
| <b>checkpoint_timeout</b>           | Maximum time between automatic checkpoints.                                      | Works with WAL-volume triggers such as max_wal_size.                                                   |
| <b>checkpoint_completion_target</b> | Spreads checkpoint writes over part of the checkpoint interval.                  | Helps smooth write I/O.                                                                                |
| <b>max_wal_size</b>                 | Soft target influencing checkpoint frequency due to WAL growth.                  | It is not a hard maximum size for pg_wal.                                                              |
| <b>min_wal_size</b>                 | Minimum WAL retained for recycling under normal checkpoint behavior.             | Helps avoid constantly creating/removing WAL segments.                                                 |
| <b>wal_keep_size</b>                | Minimum amount of past WAL to retain for standby streaming needs.                | Replication slots provide stronger consumer-specific retention semantics.                              |

| Term                  | Simple Meaning                                  | Interview / DBA Point                                         |
|-----------------------|-------------------------------------------------|---------------------------------------------------------------|
| <b>Crash Recovery</b> | Automatic WAL replay after an unclean shutdown. | PostgreSQL restores consistency before accepting normal work. |

**Quick commands**

```
SELECT pg_current_wal_lsn();
SHOW max_wal_size;
SHOW checkpoint_timeout;
SHOW wal_keep_size;
```

## 9. MVCC and Transactions

| Term                   | Simple Meaning                                                                             | Interview / DBA Point                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>MVCC</b>            | Multi-Version Concurrency Control.                                                         | Readers and writers generally avoid unnecessary blocking by using row versions and snapshots.             |
| <b>Tuple Version</b>   | Specific physical version of a logical row.                                                | UPDATE normally creates a new tuple version.                                                              |
| <b>xmin</b>            | Tuple metadata containing the inserting transaction ID.                                    | Used as part of MVCC visibility rules.                                                                    |
| <b>xmax</b>            | Tuple metadata used for deleting/updating/locking transaction information.                 | Do not oversimplify it as only 'delete XID'.                                                              |
| <b>Snapshot</b>        | Defines which transactions/tuple versions are visible.                                     | Long-lived snapshots can delay cleanup.                                                                   |
| <b>Dead Tuple</b>      | Obsolete tuple version no longer needed once visibility rules permit cleanup.              | VACUUM makes its space reusable.                                                                          |
| <b>HOT Update</b>      | Heap-Only Tuple optimization that may avoid new index entries.                             | Requires suitable conditions such as no indexed-column change and enough page space.                      |
| <b>XID</b>             | Transaction ID used in MVCC and visibility.                                                | Finite ID space makes wraparound protection essential.                                                    |
| <b>XID Wraparound</b>  | Risk from finite transaction ID space.                                                     | VACUUM freezing prevents very old tuples from becoming unsafe.                                            |
| <b>Freeze</b>          | Marks/advances old tuple transaction information for wraparound safety.                    | Autovacuum can run aggressively to prevent wraparound.                                                    |
| <b>Read Committed</b>  | Default PostgreSQL isolation level.                                                        | Each statement sees an appropriate snapshot; PostgreSQL maps Read Uncommitted behavior to Read Committed. |
| <b>Repeatable Read</b> | Stable transaction snapshot with PostgreSQL's stronger semantics.                          | Can still raise serialization-style errors in some conflicts.                                             |
| <b>Serializable</b>    | Strongest isolation; PostgreSQL can abort transactions to prevent serialization anomalies. | Applications must be prepared to retry serialization failures.                                            |

## 10. VACUUM and Routine Maintenance

| Term                        | Simple Meaning                                                                               | Interview / DBA Point                                                                       |
|-----------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>VACUUM</b>               | Makes dead tuple space reusable and performs visibility/freeze maintenance.                  | Regular VACUUM normally does not shrink the table file back to the OS.                      |
| <b>Autovacuum</b>           | Automatic VACUUM/ANALYZE subsystem.                                                          | Critical for cleanup, statistics and wraparound protection; do not casually disable it.     |
| <b>VACUUM FULL</b>          | Rewrites a table into a compact new file.                                                    | Returns space to OS but requires ACCESS EXCLUSIVE lock and extra disk space during rewrite. |
| <b>ANALYZE</b>              | Collects statistics used by the planner.                                                     | Poor/missing stats can cause cardinality errors and bad plans.                              |
| <b>VACUUM ANALYZE</b>       | Runs VACUUM and ANALYZE together.                                                            | Useful after major data change, but autovacuum normally handles routine work.               |
| <b>Bloat</b>                | Unused/excess relation space from dead versions, free space and workload churn.              | Diagnose cause before choosing VACUUM FULL or REINDEX.                                      |
| <b>REINDEX</b>              | Rebuilds an index.                                                                           | Used for index corruption, certain bloat cases, or changed index requirements.              |
| <b>Routine Reindexing</b>   | Periodic/conditional index rebuild maintenance.                                              | Not a universal fixed schedule requirement; base it on evidence.                            |
| <b>Freeze Age</b>           | Age thresholds that trigger aggressive anti-wraparound vacuuming.                            | Monitor transaction ages on high-transaction systems.                                       |
| <b>Autovacuum Threshold</b> | Basic trigger uses threshold plus scale-factor logic, with additional modern rules/settings. | Explain the formula as a concept, not the entire autovacuum algorithm.                      |

### Quick commands

```
VACUUM (VERBOSE, ANALYZE) public.actor;
```

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;
```

## 11. Query Planning, Statistics, and Indexes

| Term                       | Simple Meaning                                                    | Interview / DBA Point                                                     |
|----------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Parser</b>              | Parses SQL and builds an internal representation.                 | Syntax/name resolution is handled before planning/execution.              |
| <b>Rewriter</b>            | Applies query rewrite rules, including view/rule transformations. | Part of PostgreSQL query processing before planning.                      |
| <b>Planner / Optimizer</b> | Chooses a cost-based execution plan.                              | Uses statistics, cost parameters, indexes and relational transformations. |
| <b>Executor</b>            | Runs the selected plan.                                           | EXPLAIN ANALYZE reports actual executor results/timing.                   |
| <b>EXPLAIN</b>             | Displays estimated execution plan without normally running the    | Use it for plan shape and row/cost estimates.                             |

| Term                    | Simple Meaning                                                          | Interview / DBA Point                                                                    |
|-------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
|                         | statement.                                                              |                                                                                          |
| <b>EXPLAIN ANALYZE</b>  | Executes the statement and reports actual plan statistics.              | Be careful with DML because it really executes unless wrapped/rolled back appropriately. |
| <b>Seq Scan</b>         | Sequentially scans a relation.                                          | Not automatically bad; often correct for small tables or low-selectivity filters.        |
| <b>Index Scan</b>       | Uses an index and fetches heap tuples.                                  | Cost depends on selectivity, correlation, cache and random I/O assumptions.              |
| <b>Index Only Scan</b>  | Can return values from the index when heap visibility checks allow.     | Visibility map state is important.                                                       |
| <b>Bitmap Scan</b>      | Combines index matches into a bitmap before heap access.                | Often useful for a moderate number of scattered rows.                                    |
| <b>B-tree</b>           | Default index method for equality, range, sorting and many comparisons. | Most common general-purpose PostgreSQL index.                                            |
| <b>Hash Index</b>       | Index method for equality operators.                                    | Less general than B-tree.                                                                |
| <b>GIN</b>              | Generalized Inverted Index.                                             | Common for arrays, full-text search and suitable JSONB operators.                        |
| <b>GiST</b>             | Generalized Search Tree framework.                                      | Supports extensible search strategies for specialized data types.                        |
| <b>SP-GiST</b>          | Space-Partitioned GiST.                                                 | Useful for certain partitioned search structures.                                        |
| <b>BRIN</b>             | Block Range Index.                                                      | Excellent for huge tables where values correlate with physical block order.              |
| <b>Partial Index</b>    | Indexes only rows satisfying a predicate.                               | Can reduce size and improve targeted queries.                                            |
| <b>Expression Index</b> | Indexes an expression rather than only raw columns.                     | Useful for queries using the same expression, e.g. lower(email).                         |

#### Quick commands

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM public.actor WHERE actor_id = 10;
```

## 12. Partitioning and Database Objects

| Term                            | Simple Meaning                                              | Interview / DBA Point                                           |
|---------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Declarative Partitioning</b> | Native table partitioning framework.                        | Supported strategies: RANGE, LIST, HASH.                        |
| <b>Partitioned Table</b>        | Logical parent defining partitioning rules.                 | Storage is in child partitions, not the parent itself.          |
| <b>Partition Key</b>            | Column/expression that determines row routing.              | Choose based on query filters, lifecycle and data distribution. |
| <b>Partition Pruning</b>        | Eliminates irrelevant partitions during planning/execution. | Queries should constrain the partition key effectively.         |
| <b>Default Partition</b>        | Catches rows that do not match existing partition bounds.   | Useful operationally but should be monitored.                   |

| Term                     | Simple Meaning                                                    | Interview / DBA Point                                                                                  |
|--------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>View</b>              | Stored query definition presented like a table.                   | Usually stores no result data itself.                                                                  |
| <b>Materialized View</b> | Physically stores a query result.                                 | Needs REFRESH MATERIALIZED VIEW to update its contents.                                                |
| <b>Sequence</b>          | Independent numeric sequence generator.                           | nextval is not transactional in the sense that rolled-back transactions do not return consumed values. |
| <b>Identity Column</b>   | SQL-standard generated identity mechanism backed by a sequence.   | Prefer for new designs over legacy SERIAL convenience syntax.                                          |
| <b>Generated Column</b>  | Column computed from an expression according to PostgreSQL rules. | PostgreSQL 18 also has logical replication support considerations for generated columns.               |
| <b>Extension</b>         | Packaged way to add PostgreSQL features/objects.                  | CREATE EXTENSION installs it in a database; extension binaries may also require OS installation.       |
| <b>FDW</b>               | Foreign Data Wrapper framework for accessing external data.       | postgres_fdw is the standard PostgreSQL-to-PostgreSQL example.                                         |

### 13. Backup, Restore, and PITR

| Term                   | Simple Meaning                                                                    | Interview / DBA Point                                                            |
|------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Logical Backup</b>  | SQL/object-level backup independent of physical data-file layout.                 | Common tools: pg_dump, pg_dumpall, pg_restore, psql.                             |
| <b>Physical Backup</b> | File/block-level cluster backup consistent with PostgreSQL recovery requirements. | Common base-backup tool: pg_basebackup.                                          |
| <b>pg_dump</b>         | Logical backup of one database.                                                   | Custom/directory formats support pg_restore and parallel restore features.       |
| <b>pg_dumpall</b>      | Cluster-wide logical SQL dump including databases and global objects.             | Useful for globals such as roles/tablespaces and small environments.             |
| <b>pg_restore</b>      | Restores non-plain pg_dump archive formats.                                       | Supports object selection, parallelism and restore ordering.                     |
| <b>pg_basebackup</b>   | Takes a physical base backup from a running server.                               | Used for standby creation, PITR and PostgreSQL incremental backup workflows.     |
| <b>Backup Manifest</b> | Metadata/checksum manifest for base backups.                                      | pg_verifybackup validates a backup against its manifest.                         |
| <b>pg_verifybackup</b> | Verifies a PostgreSQL base backup.                                                | Verification is not a substitute for regular restore testing.                    |
| <b>PITR</b>            | Point-in-Time Recovery using a base backup plus required WAL.                     | Recover to time, LSN, transaction ID, restore point, or other supported targets. |
| <b>WAL Archiving</b>   | Copies completed WAL segments to durable archive storage.                         | Core settings: archive_mode and archive_command/archive_library.                 |
| <b>restore_command</b> | Command used during recovery to fetch archived WAL.                               | Must be reliable and correctly handle missing/unavailable files.                 |

| Term                      | Simple Meaning                                                                    | Interview / DBA Point                                                   |
|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Timeline</b>           | Branch of WAL history created by promotion/recovery events.                       | Important when selecting recovery history after failover/PITR.          |
| <b>Incremental Backup</b> | Physical backup containing changed blocks relative to a prior backup.             | PostgreSQL 18 uses WAL summaries for native incremental backup support. |
| <b>WAL Summary</b>        | Summary of blocks changed by WAL over a range.                                    | Generated by the WAL summarizer and used to identify changed blocks.    |
| <b>pg_combinebackup</b>   | Reconstructs a synthetic full backup from a full + dependent incremental backups. | All required dependent backups in the chain must be available.          |

## 14. Physical and Logical Replication

| Term                             | Simple Meaning                                                                        | Interview / DBA Point                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Primary</b>                   | Server accepting writes and generating WAL.                                           | A standby can be promoted if the primary fails.                                           |
| <b>Standby</b>                   | Server continuously recovering WAL from upstream/archive.                             | Hot standby can serve read-only queries while in recovery.                                |
| <b>Streaming Replication</b>     | Continuously streams WAL records from upstream to standby.                            | Implemented using WAL sender and WAL receiver processes.                                  |
| <b>Physical Replication</b>      | Replicates physical WAL changes at cluster level.                                     | Primary and standby generally need compatible major-version/physical format expectations. |
| <b>Asynchronous Replication</b>  | Primary commit does not wait for standby acknowledgement.                             | Lower latency but potential data loss at failover.                                        |
| <b>Synchronous Replication</b>   | Commits wait according to configured synchronous standby acknowledgement rules.       | Reduces data-loss risk but adds latency/availability dependency.                          |
| <b>Replication Lag</b>           | Difference between primary progress and standby receive/write/flush/replay positions. | Lag is not one number; inspect the stage and application requirement.                     |
| <b>Replication Slot</b>          | Retains WAL/resources required by a replication consumer.                             | Inactive slots can cause unlimited/large WAL retention unless bounded/configured.         |
| <b>Physical Replication Slot</b> | Slot for physical streaming consumer.                                                 | Helps prevent required WAL from being removed prematurely.                                |
| <b>Logical Replication Slot</b>  | Slot supporting logical decoding/replication consumer state.                          | Can retain WAL and catalog xmin; monitor carefully.                                       |
| <b>Publisher</b>                 | Database containing publications.                                                     | Logical replication starts from publication definitions.                                  |
| <b>Publication</b>               | Defines tables and change types made available for logical replication.               | Can use row filters and column lists where supported.                                     |
| <b>Subscriber</b>                | Database receiving logical changes.                                                   | Contains subscription objects and apply workers.                                          |
| <b>Subscription</b>              | Subscriber-side object connecting to publications.                                    | Can perform initial table copy then stream ongoing changes.                               |

| Term                    | Simple Meaning                                             | Interview / DBA Point                        |
|-------------------------|------------------------------------------------------------|----------------------------------------------|
| <b>Logical Decoding</b> | Transforms WAL into a logical stream of row-level changes. | Used by logical replication and CDC tooling. |
| <b>Output Plugin</b>    | Formats logical decoding output.                           | Built-in logical replication uses pgoutput.  |

## 15. High Availability and Recovery Concepts

| Term                          | Simple Meaning                                                                                           | Interview / DBA Point                                                                                 |
|-------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>High Availability (HA)</b> | Architecture designed to reduce downtime after failures.                                                 | PostgreSQL provides replication/promotion building blocks; automated orchestration is often external. |
| <b>Failover</b>               | Unplanned transition from failed primary to another server.                                              | Requires client redirection and split-brain protection.                                               |
| <b>Switchover</b>             | Planned role transition between healthy servers.                                                         | Used for maintenance/testing with controlled risk.                                                    |
| <b>Promotion</b>              | Ends standby recovery and makes the server writable.                                                     | Creates a new timeline.                                                                               |
| <b>RPO</b>                    | Recovery Point Objective - acceptable data loss.                                                         | Synchronous design can reduce RPO, but architecture and failure mode matter.                          |
| <b>RTO</b>                    | Recovery Time Objective - acceptable service outage duration.                                            | Depends on detection, orchestration, promotion, DNS/VIP/pooler and application recovery.              |
| <b>Split Brain</b>            | Multiple nodes incorrectly accept writes as primary.                                                     | Can create irreconcilable divergence.                                                                 |
| <b>Fencing</b>                | Prevents the old primary from continuing to serve writes after failover.                                 | Critical in automated HA designs.                                                                     |
| <b>pg_rewind</b>              | Synchronizes a former primary/diverged cluster with the new primary timeline when prerequisites are met. | Often faster than rebuilding a full standby.                                                          |
| <b>Cascading Replication</b>  | Standby streams WAL to another downstream standby.                                                       | Reduces direct load/connections on primary and helps topology design.                                 |

## 16. Monitoring and Troubleshooting

| Term                       | Simple Meaning                                         | Interview / DBA Point                                                                |
|----------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>pg_stat_activity</b>    | Shows sessions, states, queries, timestamps and waits. | First-stop view for active sessions and long-running transactions.                   |
| <b>pg_locks</b>            | Shows lock state and lock holders/waiters.             | Join with pg_stat_activity for blocking analysis.                                    |
| <b>pg_stat_database</b>    | Database-level cumulative activity statistics.         | Useful for commits/rollbacks, blocks, tuples, temp files and session-level counters. |
| <b>pg_stat_user_tables</b> | User-table activity and maintenance                    | Check scans, tuples, vacuum/analyze activity and                                     |

| Term                            | Simple Meaning                                                                   | Interview / DBA Point                                                                            |
|---------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
|                                 | statistics.                                                                      | dead tuples.                                                                                     |
| <b>pg_stat_user_indexes</b>     | User-index usage statistics.                                                     | Use carefully before dropping 'unused' indexes; consider resets, rare workloads and constraints. |
| <b>pg_stat_replication</b>      | Primary-side physical replication status.                                        | Inspect state, LSNs, sync_state and lag-related positions.                                       |
| <b>pg_stat_wal_receiver</b>     | Standby WAL receiver status.                                                     | Useful for upstream connection and received-LSN troubleshooting.                                 |
| <b>pg_stat_archiver</b>         | WAL archiver success/failure statistics.                                         | Archive failures are a storage and recoverability risk.                                          |
| <b>pg_stat_checkpointer</b>     | Checkpoint statistics.                                                           | Modern PostgreSQL separates checkpointer stats from pg_stat_bgwriter.                            |
| <b>pg_stat_bgwriter</b>         | Background writer statistics.                                                    | Use with checkpointer and I/O statistics for write-path analysis.                                |
| <b>pg_stat_io</b>               | Cluster-wide I/O statistics by backend/object/context categories.                | PostgreSQL 18 provides richer byte-level I/O counters.                                           |
| <b>pg_stat_statements</b>       | Extension that aggregates statement planning/execution statistics.               | Requires configuration and CREATE EXTENSION in databases where the view is used.                 |
| <b>Wait Event</b>               | Reason a backend is waiting.                                                     | Common types include Lock, LWLock, IO, Client, IPC, Timeout, Activity, Extension.                |
| <b>Blocking</b>                 | Session waits for incompatible lock/resource held by another session.            | Find root blocker, not just blocked sessions.                                                    |
| <b>Deadlock</b>                 | Circular wait between transactions.                                              | PostgreSQL detects it and aborts one transaction.                                                |
| <b>Long-Running Transaction</b> | Transaction open for a long time.                                                | Can delay vacuum cleanup, retain snapshots, hold locks and increase bloat.                       |
| <b>Temporary Spill</b>          | Sort/hash/other operation writes temporary files because memory is insufficient. | Investigate work_mem, query plan, data volume and log_temp_files.                                |
| <b>Archive Failure</b>          | Completed WAL cannot be archived successfully.                                   | Can grow pg_wal and compromise PITR continuity.                                                  |
| <b>Checkpoint Pressure</b>      | Checkpoint-related writes create excessive I/O/latency.                          | Review checkpoint stats, WAL growth, storage latency and configuration.                          |

**Quick commands**

```
SELECT pid, username, datname, state, wait_event_type, wait_event, query FROM pg_stat_activity;
SELECT * FROM pg_locks;
```

## 17. Core DBA Utilities

| Term          | Simple Meaning                  | Interview / DBA Point                                       |
|---------------|---------------------------------|-------------------------------------------------------------|
| <b>psql</b>   | PostgreSQL command-line client. | Know connection syntax, meta-commands and script execution. |
| <b>pg_ctl</b> | Native server control utility.  | Start, stop, restart, reload, status and promote functions. |

| Term                         | Simple Meaning                                                               | Interview / DBA Point                                                                      |
|------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>pg_isready</b>            | Checks whether a PostgreSQL server is accepting connections.                 | Useful in scripts and health checks; it does not validate application semantics.           |
| <b>createdb / dropdb</b>     | Command-line database creation/removal utilities.                            | Wrappers around SQL database management operations.                                        |
| <b>createuser / dropuser</b> | Command-line role management utilities.                                      | PostgreSQL calls identities roles even when using createuser.                              |
| <b>vacuumdb</b>              | Command-line VACUUM/ANALYZE utility.                                         | Supports parallel and staged analyze options; PostgreSQL 18 includes --missing-stats-only. |
| <b>reindexdb</b>             | Command-line REINDEX utility.                                                | Automates rebuilding indexes across objects/databases.                                     |
| <b>pgbench</b>               | Benchmark/workload generation utility.                                       | Good for repeatable testing, not a universal production sizing tool.                       |
| <b>pg_amcheck</b>            | Checks supported relation/index structures for corruption/inconsistency.     | Complements checksums and backup/restore validation.                                       |
| <b>pg_checksums</b>          | Enables/disables/verifies data checksums on an offline cluster as supported. | PostgreSQL 18 initdb enables checksums by default.                                         |
| <b>pg_receivewal</b>         | Streams WAL to a local archive-like destination.                             | Useful for continuous WAL collection.                                                      |
| <b>pg_waldump</b>            | Displays WAL record contents for low-level analysis.                         | Advanced troubleshooting/forensics utility.                                                |
| <b>pg_walsummary</b>         | Displays WAL summary files.                                                  | Relevant to native incremental backup workflows.                                           |

## 18. Upgrades and Lifecycle

| Term                            | Simple Meaning                                                                 | Interview / DBA Point                                                                   |
|---------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Minor Upgrade</b>            | Upgrade within the same major version, e.g. 18.5 to 18.6.                      | Does not require pg_upgrade; install updated binaries/packages and restart as required. |
| <b>Major Upgrade</b>            | Upgrade between major versions, e.g. 16 to 18.                                 | Requires migration such as pg_upgrade, dump/restore, or replication.                    |
| <b>pg_upgrade</b>               | Upgrades a cluster to a later major version without full logical dump/restore. | Always use the pg_upgrade executable from the new PostgreSQL version.                   |
| <b>pg_upgrade --check</b>       | Compatibility/preflight mode that does not change cluster data.                | Run before the outage/cutover and fix every reported blocker.                           |
| <b>Old/New Binary Directory</b> | Directories containing old and new PostgreSQL executable sets.                 | pg_upgrade needs both to test and operate correctly.                                    |
| <b>Old/New Data Directory</b>   | Source and destination cluster storage/config directories.                     | Confirm actual paths with PostgreSQL configuration/tools, not assumptions.              |
| <b>Copy Mode</b>                | Default pg_upgrade transfer mode that copies relation files.                   | Uses more time/storage but leaves the old data files independent.                       |

| Term                           | Simple Meaning                                                                | Interview / DBA Point                                                                     |
|--------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Link Mode</b>               | Hard-links user data files where supported.                                   | Very fast/space efficient, but old cluster becomes unsafe to use once the new one starts. |
| <b>Clone Mode</b>              | Filesystem clone/reflink-based transfer where supported.                      | Fast/space efficient while preserving old cluster independence better than link mode.     |
| <b>Swap Mode</b>               | Swaps compatible file/directory locations under strict filesystem conditions. | Can be very fast but has rollback/access implications.                                    |
| <b>Extension Compatibility</b> | Extensions must be available and binary-compatible with the new version.      | pg_upgrade cannot guarantee every external module is compatible.                          |
| <b>Post-Upgrade Analyze</b>    | Refreshes optimizer statistics after major upgrade.                           | Essential for stable plans; staged analysis can help service return sooner.               |

### Quick commands

```
pg_upgrade --check ...
vacuumdb --all --analyze-in-stages
```

## Official PostgreSQL References

All core PostgreSQL terminology in this document is aligned to PostgreSQL 18.6 official documentation. External tools such as PgBouncer are explicitly identified as non-core.

- **PostgreSQL 18.6 Documentation:** <https://www.postgresql.org/docs/18/>
- **Part III - Server Administration:** <https://www.postgresql.org/docs/18/admin.html>
- **Chapter 18 - Server Setup and Operation:** <https://www.postgresql.org/docs/18/runtime.html>
- **Chapter 19 - Server Configuration:** <https://www.postgresql.org/docs/18/runtime-config.html>
- **Chapter 20 - Client Authentication:** <https://www.postgresql.org/docs/18/client-authentication.html>
- **Chapter 24 - Routine Database Maintenance:** <https://www.postgresql.org/docs/18/maintenance.html>
- **Chapter 25 - Backup and Restore:** <https://www.postgresql.org/docs/18/backup.html>
- **Chapter 26 - High Availability, Load Balancing, and Replication:** <https://www.postgresql.org/docs/18/high-availability.html>
- **Chapter 27 - Monitoring Database Activity:** <https://www.postgresql.org/docs/18/monitoring.html>
- **Chapter 28 - Reliability and WAL:** <https://www.postgresql.org/docs/18/wal.html>
- **Chapter 29 - Logical Replication:** <https://www.postgresql.org/docs/18/logical-replication.html>
- **pg\_upgrade:** <https://www.postgresql.org/docs/18/pgupgrade.html>
- **pg\_combinebackup:** <https://www.postgresql.org/docs/18/app-pgcombinebackup.html>
- **PostgreSQL 18 Release Notes:** <https://www.postgresql.org/docs/18/release-18.html>

**Interview note:** Terminology has been simplified for interview revision. Where PostgreSQL has nuanced internal behavior, the wording avoids treating simplified descriptions as exact implementation details.